

Advanced Backend

why not use a single backend server?

1. Traffic limit (scalability)

A single server can only handle **limited requests per second**.

Example:

2. Single Point of Failure

If you only have one backend:

User → Backend Server

If that server crashes:

💥 **Your whole app is down.**

3. Zero Downtime Deployments

Imagine you deploy new code.

With one backend:

Stop server → deploy → start server

Your app is **down during deployment**.

4. Geographic latency

Users in different regions experience delays.

Example:

- Server in **India**
- User in **USA**

Request takes longer.

So companies run servers in **multiple regions**.

Ways to communicate in backend

Backend Communication

- └─ Synchronous (wait for response)
- └─ Asynchronous (fire and continue)

1. HTTP / REST APIs

Most common method.

Service A → HTTP Request → Service B

Example:

Order Service → POST /create-payment → Payment Service

Mental model

Just like a browser calling an API, but it's server to server.

Pros

- Simple
- Standard
- Easy debugging
- Works everywhere

Cons

- Slower than binary protocols
- Tight coupling
- If Service B is down → Service A fails

Used in

- Microservices
- Public APIs
- Internal APIs

Interview line:

Services communicate via REST APIs over HTTP using JSON.

2. WebSockets — Persistent connection

Client ⇄ WebSocket connection ⇄ Server

Once connected:

- both sides can send messages anytime
- connection stays open

Used for:

- Chat
- Live notifications
- Trading dashboards
- Multiplayer games

Example:

User message → WebSocket server → broadcast to other users

Async communication

1. Message Queues

A queue is like a task line.

Producer → Queue → Worker

Example:

Order Service → Queue → Email Worker

When an order is placed:

1. Order service sends message to queue
2. Worker takes message
3. Worker processes it
4. Message is removed

Important part:

👉 Usually only ONE consumer processes each message.

How it works internally

Producer → Queue → Consumer 1

Workers compete for messages.

Flow

1. **Producer pushes message**
2. **Message stored in queue**
3. **Worker pulls message**
4. **Worker acknowledges (ACK)**
5. **Message removed**

If worker crashes:

Queue → re-delivers message

This makes queues reliable.

Where queues are used

Queues are great for background jobs.

Examples:

- **sending emails**
- **video processing**
- **image resizing**
- **report generation**
- **payment processing retries**
-

Example architecture:

User → API Server



Queue



Workers

The API responds immediately, workers handle heavy work later.

Popular queue tools

- RabbitMQ
- AWS SQS
- Redis queues (BullMQ)
- ActiveMQ

2. Pub/Sub (Publish–Subscribe)

Mental model

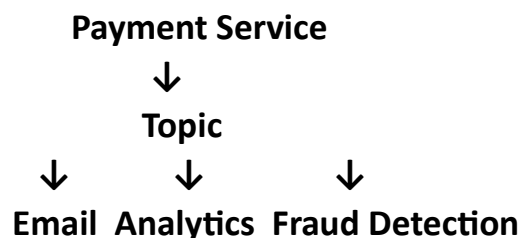
Instead of a task line, think of a radio broadcast.

Publisher → Topic → Subscribers

Example:

Payment Service → "Payment Completed" event

Multiple services receive it.



Each subscriber gets the same event.

How it works internally

Messages are sent to a topic.

Publisher → Topic



Each subscriber processes its own copy.

Unlike queues:

👉 Message is not consumed once globally.

Each subscriber decides what to do.

Example event

```
{  
  "event": "payment_completed",  
  "orderId": 8491,  
  "amount": 1200  
}
```

Subscribers react differently:

- Email service → send receipt
- Analytics → update revenue
- Fraud service → run checks

Publisher does not know who consumes the event.

This is called loose coupling.

Stateful vs Stateless backends

1. What is “State”?

State = stored information about a client/session between requests.

Example state:

- login session
- shopping cart
- user progress
- WebSocket connection
- game session

Example:

User logs in → server remembers “this user is authenticated.”

That remembered info = state.

2. Stateless Backend

Definition

A stateless server does not store client state between requests.

Every request contains all the information needed.

Mental model

Request 1 → Server

Server processes → forgets everything

Request 2 → Server

Server processes → forgets everything

Server treats every request like it's brand new.

Example

HTTP APIs are usually stateless.

Client → GET /profile

Authorization: Bearer JWT

The request includes:

- authentication token
- required data

Server verifies the token and responds.

It does not remember anything about the client.

Architecture

Clients



Load Balancer



Server 1

Server 2

Server 3

Any server can handle the request.

Because no session data is stored in servers.

Benefits

1. Easy scaling 10 servers → 100 servers

Add more servers easily. No session syncing needed.

2. High reliability

If a server crashes: Nothing breaks. Request goes to active servers

3. Simple load balancing

Load balancer can route requests randomly.

Where stateless systems are used

- REST APIs
- microservices
- cloud architectures
- serverless systems

Example companies:

- Netflix
- Amazon
- Stripe APIs

Stateful Backend

A stateful server stores client session information internally.

The server remembers the client between requests.

Mental model

Client connects → Server stores session

Client request → Server uses stored session

Server must remember:

- who the user is
- what they were doing

Example

Classic login sessions.

User logs in

Server stores session in memory

Next request:

Client → sends session ID cookie

Server → finds session in memory

Server uses stored data.

Architecture

Clients



Load Balancer



Server 1 (session A)

Server 2 (session B)

Now the problem appears.

If request goes to wrong server:

User session stored in Server 1

Request goes to Server 2

Server 2 does not know the session.

User appears logged out.

4. Sticky Sessions (Stateful Fix)

Load balancer forces user to same server.

User → Server 1

User → Server 1

User → Server 1

This is called session affinity or sticky sessions.

But it creates problems.

Problem 1: Scaling becomes hard

Server 1 might get overloaded.

100k users → Server 1

10 users → Server 2

Load imbalance.

Problem 2: Server crash

If Server 1 crashes:

User sessions lost

All users logged out.

5. Hybrid Approach (Real-world solution)

Modern systems keep servers stateless, but move state to external storage.

Example:

Clients



Load Balancer



Stateless Servers



Redis / Database

Session data stored in:

- **Redis**
- **database**
- **distributed cache**

Example:

User logs in

Session stored in Redis

Any server can access it.

7. Stateful systems examples

These systems must store state.

Examples:

- **WebSockets**
- **multiplayer games**
- **database connections**
- **video streaming sessions**

Example WebSocket:

Client $\rightleftharpoons$ WebSocket Server

Connection must stay on the same server.

That server stores the connection.

So WebSocket servers are stateful.

Stateless

A stateless system does not store client session data between requests. Each request contains all required information, allowing any server to handle it independently.

Stateful

A stateful system stores client session information on the server, meaning requests from the same client must often be routed to the same server.

So why use stateful architecture when we know it is pain..?

Stateful services are needed when a system must remember information between requests or maintain ongoing context. For example, databases store persistent data like user records and transactions, WebSocket servers maintain active client connections for real-time apps like chat, and streaming systems track processing offsets. Another case is real-time systems like live games or trading platforms, where data must be kept in memory to process updates extremely fast, since reading and writing to a database for every action would be too slow. While stateless services are easier to scale, stateful services are necessary to manage persistent data, real-time interactions, and high-speed in-memory processing.

Singleton Pattern

Why do we need the Singleton pattern?

The Singleton pattern is used when multiple parts of an application need to share the same state or resource, and creating multiple instances would cause inconsistency, conflicts, or wasted resources.

In stateful systems, different components may need to read and update the same shared state, so having multiple independent instances can break the system.

Singleton ensures there is one consistent source of truth inside the application.

Problem in stateful architecture

Imagine multiple instances managing the same state.

Example: In-memory cache

Without Singleton:

Service A → Cache instance 1

Service B → Cache instance 2

Service C → Cache instance 3

Now data becomes inconsistent.

Cache1: userBalance = 500

Cache2: userBalance = 300

Different parts of the system see different state.

This causes data inconsistency bugs.

How Singleton fixes it

Singleton ensures only one instance manages that state.

Services



Singleton Cache



Shared State

Now everyone reads and writes to the same memory.

Another real backend example

Database connection manager

Without Singleton:

Request 1 → new DB connection

Request 2 → new DB connection

Request 3 → new DB connection

This can exhaust database connections.

With Singleton:

Requests



Singleton Connection Pool



Database

One shared pool handles everything.

What problem Singleton solves

Singleton solves these problems in stateful systems:

1. State inconsistency

Multiple instances may hold different state.

2. Resource overuse

Expensive resources like DB connections could be created repeatedly.

3. Coordination issues

Different parts of the system might behave differently if they don't share the same state.

2. Classic Singleton Pattern

This is the textbook interview implementation.

```
class Singleton {  
  constructor() {  
    if (Singleton.instance) {  
      return Singleton.instance;  
    }  
  
    this.data = "I am the only instance";  
    Singleton.instance = this;  
  }  
  
  getData() {  
    return this.data;  
  }  
}
```

```
const obj1 = new Singleton();  
const obj2 = new Singleton();
```

```
console.log(obj1 === obj2); // true
```

Even if you call new Singleton() multiple times, it returns the same object.

3. Static method Singleton (cleaner)

This is a very common interview answer.

```
class Singleton {  
  static instance;  
  
  constructor() {  
    if (Singleton.instance) {
```

```
throw new Error("Use getInstance()");  
}  
}
```

```
static getInstance() {  
  if (!Singleton.instance) {  
    Singleton.instance = new Singleton();  
  }  
  return Singleton.instance;  
}  
}
```

```
const obj1 = Singleton.getInstance();  
const obj2 = Singleton.getInstance();
```

```
console.log(obj1 === obj2); // true
```